Trabajo Práctico – Programación Imperativa

Arimaa

Introducción a la Programación – Segundo cuatrimestre de 2026

Fecha límite de entrega:
viernes 30 de octubre 23:59hs

Introducción

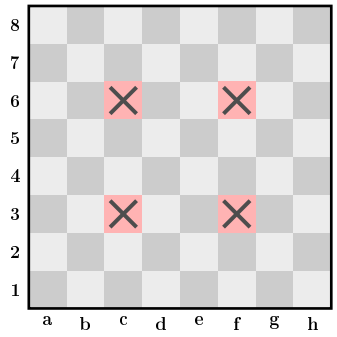
El objetivo de este trabajo es aplicar los conceptos de programación imperativa vistos en la materia para resolver ejercicios utilizando el lenguaje de programación Python. El trabajo consiste en implementar el juego **Arimaa**[1]. Se trata de un juego de tablero diseñado por Omar Syed en 2002, con tres objetivos: (1) que sea posible jugar usando el tablero y las piezas de un juego de ajedrez común, (2) que las reglas sean suficientemente sencillas como para poder enseñarle a jugar a su hijo de cuatro años de edad, y (3) que el juego resulte "difícil" desde el punto de vista computacional.

Este TP consta de 2 partes:

- Parte I: resolver los ejercicios que se presentan en este documento. La fecha límite para resolverlo es el día viernes 30 de octubre. Deben contemplar todas las condiciones detalladas en este documento.
- Parte II: el día lunes 02 de noviembre deben asistir *todos* los integrantes del grupo al taller de su turno. Durante ese taller, deberán implementar funcionalidad extra que será dada en el momento. Al finalizar el taller, deben enviar el TP completo (lo realizado en la Parte I más lo implementado durante el taller). Al terminar el taller se deshabilitará el envío del TP.

Reglas del juego

Arimaa es un juego de **dos jugadores** que se juega sobre un tablero de 8×8 casillas. Las filas se enumeran de `1` a `8` y las columnas de `a` a `h`. Un jugador controla piezas **doradas** y el oponente controla piezas **plateadas**. El tablero tiene cuatro casillas **trampa**: `c3`, `c6`, `f3` y `f6`. El tablero comienza vacío:



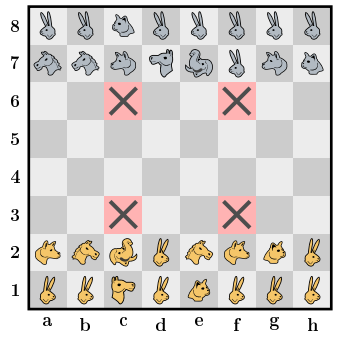
Cada jugador dispone de dieciséis piezas; de mayor a menor **fuerza**: un elefante (🐘/🐘), un camello (🐪/🐪), dos caballos (🐴/🐴), dos perros (🐶/🐶), dos gatos (🐱/🐱), y ocho conejos (🐰/🐰). Cada uno de los seis tipos de piezas se llaman **figuras**. Las piezas más fuertes pueden **empujar** y **arrastrar**[2] a las piezas más débiles del oponente. Además, las piezas más fuertes **congelan**[3] a las piezas adyacentes del oponente.

El juego tiene dos etapas. La primera etapa del juego corresponde a la ubicación de las piezas, y la segunda etapa es el desarrollo en sí.

[1] Técnicamente, se trata de una ligera variante del juego.
[2] *Push* y *pull*.
[3] *Freeze*.

Primera etapa: ubicación de las piezas

El tablero comienza vacío. Comienza el jugador que controla las piezas doradas. Los jugadores se alternan ubicando una por una sus piezas en alguna casilla vacía. En esta etapa las piezas no se pueden mover. Las piezas doradas se pueden ubicar únicamente en las filas 1 y 2, mientras que las piezas plateadas se pueden ubicar únicamente en las filas 7 y 8. Por ejemplo, al final de la primera etapa el tablero se podría ver del siguiente modo:



Segunda etapa: desarrollo del juego

Una vez ubicadas todas las piezas, los jugadores se alternan haciendo **jugadas**. Comienza el jugador que controla las piezas doradas. El **objetivo** del juego es **ubicar un conejo en la fila inicial del oponente**, es decir, el jugador que controla las piezas doradas debe ubicar un conejo en la fila 8, mientras que el jugador que controla las piezas plateadas debe ubicar un conejo en la fila 1.

Cada **jugada** consta de varios **movimientos**. Además, cada **movimiento** de un jugador consta de uno o dos **pasos**:

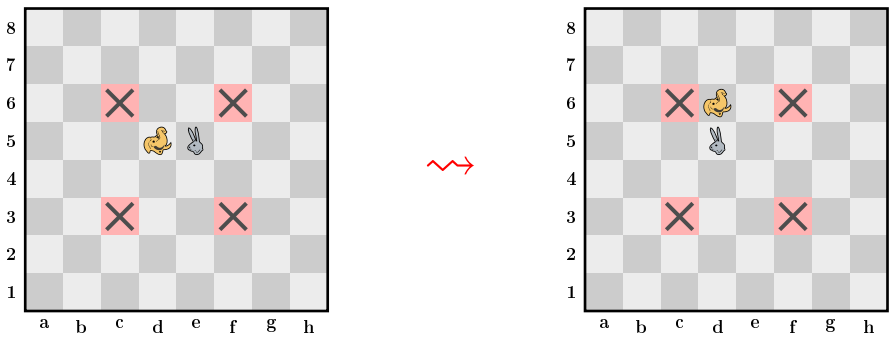
- Un movimiento que sólo mueve una pieza propia cuenta como un único paso.
- Un movimiento que mueve una pieza propia y además mueve una pieza enemiga (arrastrándola o empujándola) cuenta como dos pasos.

En cada jugada, el jugador debe hacer **al menos un movimiento**, pero puede dar **a lo sumo** 4 **pasos**.

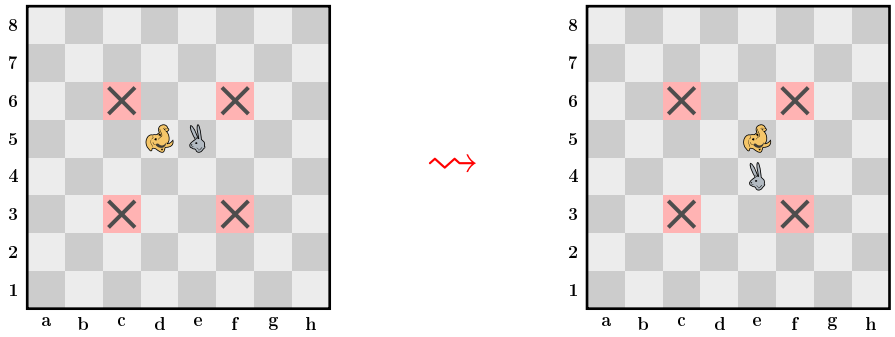
En cada movimiento, el jugador mueve una pieza propia una casilla hacia alguna de las cuatro direcciones ortogonales (norte, sur, este u oeste). Cabe observar que, a diferencia del ajedrez, **todas las piezas se mueven de la misma manera**. La única excepción es que **los conejos no pueden retroceder** (como los peones en el ajedrez). Cabe observar también que, en una misma jugada, el jugador tiene muchas opciones: puede elegir hacer varios movimientos con la misma pieza o puede elegir mover cuatro piezas diferentes. Además, una misma pieza puede cambiar de dirección a lo largo de sus movimientos.

Los movimientos de las piezas están sujetos a las siguientes reglas:

- Cuando una pieza se mueve, puede elegir **arrastrar** a una pieza adyacente del oponente, siempre que dicha pieza sea más débil. Para ello, primero se mueve la pieza más fuerte hacia alguna casilla vecina que se encuentre vacía, y a continuación se mueve la pieza más débil del oponente hacia la casilla original. Por ejemplo, el elefante dorado se puede mover hacia el norte, arrastrando al conejo plateado hacia el oeste:



- Cuando una pieza se mueve, puede elegir **empujar** a una pieza adyacente del oponente, siempre que dicha pieza sea más débil. Para ello, primero se mueve la pieza del oponente hacia alguna casilla vecina que se encuentre vacía, y a continuación se mueve la pieza más fuerte hacia la casilla donde se encontraba el oponente. Por ejemplo, el elefante dorado se puede mover hacia el este, empujando al conejo plateado hacia el sur:



- Se dice que una pieza **congela** a las piezas del oponente adyacentes a ella, siempre que sean más débiles. Una pieza congelada no puede hacer ningún movimiento (aunque sí puede ser empujada o arrastrada). Sin embargo, si una pieza congelada tiene una pieza vecina del color propio, se "descongela" y puede moverse libremente.
- Si al final de un movimiento hay alguna pieza propia o del oponente ubicada en alguna de las casillas **trampa**, y no tiene piezas vecinas de su propio color, dicha pieza se retira inmediatamente del juego.

  Notar que la pieza que se retira puede ser la pieza que el jugador acaba de mover, o una pieza del oponente que acaba de ser arrastrada o empujada, pero también puede ser una pieza que se encontraba previamente en la casilla trampa y que (como consecuencia del movimiento) deja de tener vecinos de su propio color.

Condiciones de finalización

El juego finaliza en cualquiera de los siguientes casos:

- Si al final de una jugada hay un conejo en la fila inicial del oponente, gana el jugador que controla dicho conejo.
- Si al final de una jugada hay algún jugador que no tiene conejos, gana el oponente de dicho jugador.

Nota (I). Estas dos reglas presentan algunas ambigüedades; consultar la especificación formal para subsanarlas.

Nota (II). Puede ocurrir que el juego quede "trabado" debido a que el jugador a quien le toca mover no puede hacer ningún movimiento (p.ej. porque todas sus piezas están congeladas o bloqueadas). En esta variante del juego, llegado el caso de dicha situación, el juego simplemente se traba y no se declara ningún ganador.

Archivos

En el campus de la teórica pueden encontrar un proyecto arimaa.zip el cual contiene los siguientes archivos:

- `definiciones.py` : contiene definiciones de tipos, funciones y procedimientos ya implementados. No deben modificar este archivo.
- `arimaa.py` : contiene la lógica del juego. Los procedimientos y funciones a implementar están declarados en el archivo, pero su funcionalidad no se encuentra implementada. Deben **modificar este archivo** completando toda la implementacion. **No deben cambiarles los nombres ni los tipos a las funciones ya declaradas**. Sí pueden (y deben) agregar los procedimientos y funciones auxiliares que consideren necesarios para implementar las funcionalidades pedidas de forma modular.
- `tests.py` : contiene algunos ejemplos de casos de test. Deben **modificar este archivo** agregando sus propios casos de test, para validar la funcionalidad implementada y alcanzar al menos el 90% de cubrimiento de líneas y ramas.
- `interfaz.py`: contiene la implementación de una interfaz gráfica del juego, que usa la lógica implementada por ustedes. La interfaz gráfica sólo funcionará correctamente una vez que toda la funcionalidad del TP se encuentre implementada. De lo contrario, pueden experimentar comportamientos difíciles de predecir.
- `cubrimiento.py`: es un *script* para obtener cubrimiento de líneas y ramas de forma simple. No deben modificar este archivo.
- Otros archivos auxiliares:
  - `README.txt`: resumen de algunos comandos útiles para chequear los tipos, ejecutar los casos de test, evaluar el cubrimiento y ejecutar la interfaz gráfica.
  - `piezas/*.png`: imágenes de las piezas que se usan en la interfaz gráfica.
  - `ejemplo_tablero/tablero.txt`: ejemplo de un tablero.

Pautas de Entrega

Para la entrega del trabajo práctico se deben tener en cuenta las siguientes consideraciones:

- El trabajo se debe realizar en grupos de 3 o 4 estudiantes *de forma obligatoria*.
  No se aceptarán trabajos de menos ni más integrantes.
  Ver aviso en el campus de la materia donde indica el link para registrar los grupos.
- Se debe implementar un conjunto de casos de test para todos los ejercicios (no es obligatorio para los procedimientos y funciones auxiliares que definan ustedes).
- Los programas deben pasar con éxito los casos de test escritos por ustedes (en el archivo `tests.py`) y un conjunto de casos de test secretos provistos por la cátedra que se pueden correr a través del campus de la materia (en el cuestionario "Validar Tests TP").
- El archivo con el código fuente debe tener nombre `arimaa.py`.
- No está permitido alterar los nombres de los procedimientos y funciones a implementar ni los tipos de datos. Deben mantenerse tal cual como descargan.
- Pueden definir todos los procedimientos y funciones auxiliares que requieran.
- Deben tipar las variables, las funciones y los parámetros que definan.
- No está permitido utilizar técnicas no vistas en clase para resolver los ejercicios. En el campus, pueden encontrar un listado de todos los procedimientos y funciones válidos a usar, así como también los operadores y otras construcciones del lenguaje (como alternativas y repeticiones). Si usan alguna construcción fuera de esa lista en algún ejercicio, se desaprobará el TP y deberán re-entregar automáticamente.
- Recordar que no se acepta un número de integrantes diferente a 3 o 4, pero aceptamos que queden menos en caso de que alguien abandone la materia. Esto deben comentarlo en el formulario cuando envían el TP. Las personas que abandonen no pueden entregar de forma individual.
- El TP no se califica con nota numérica; sólo se califica como Aprobado/Desaprobado.
- Todos los integrantes del grupo deben poder responder preguntas tanto de la especificación de cualquier ejercicio, como a la forma puntual de cómo lo implementaron y los tests que definieron.

Se evaluarán las siguientes características:

- **Corrección**: todos los ejercicios deben estar bien resueltos, es decir, deben respetar su especificación.
- **Declaratividad**: los nombres de las variables, parámetros, procedimientos y funciones que se definan deben ser apropiados para el problema que se resuelve. Los nombres declarativos ayudan a la legibilidad del código. La estructura del código también debería comunicar de manera acorde la intención. Para ello, se espera que eviten el uso de `break` o de múltiples `return`s así como también se espera que eviten manifestar síntomas del síndrome del miedo al booleano.
- **Modularización**: evitar repetir código innecesariamente y usar adecuadamente los procedimientos y funciones definidos previamente (por el enunciado o por ustedes mismos). Pueden agregar más procedimientos y funciones para separar el código en tareas específicas y facilitar su lectura. Se recomienda evitar el código con niveles profundos de identación; si el código tiene más de tres niveles de anidamiento, probablemente se pueda reescribir usando una subtarea.
- **Documentación**: Deben usar docstrings. Recuerden que un comentario con # no es documentación. buscamos que su código empiece a cumplir un estándar básico de calidad. La documentación debe describir **qué** hace cada función, y no **cómo** lo hace. En caso de tenerlas, cuáles son sus precondiciones. Los tests no requieren docstrings, pero sí respeten el estándar de legibilidad con buenos nombres de funciones y variables. Si lo consideran necesario agreguen aclaraciones de comentarios para el equipo de programación que lea su código con decisiones de diseño e implementación. Es obligatorio contar al menos con esta documentación para los 7 problemas que deben implementar. Para las funciones auxiliares que implementen, es aconsejable. Un ejemplo podría ser el siguiente:

```
def esBisiesto(año: int) -> bool:
    """
    Indica si el año *año* es bisiesto.
```

```
    PRE: True

    Args:
        año (int): El número de año para el cual se indica si es bisiesto.

    Returns:
        True si el año *año* es bisiesto y False si no.
    """
    return (año % 4 == 0 and año % 100 != 0) or (año % 400 == 0)
```

- **Testing**: todos los ejercicios del enunciado (sólo los procedimientos y funciones que se piden, no los procedimientos y funciones auxiliares que implementen) deben tener sus propios casos de test que pasen correctamente. Un ejercicio sin casos de test se considera *no resuelto*. Los casos de test deben cubrir distintos escenarios de uso de los procedimientos y funciones y **deben tener una cobertura de al menos 90% de líneas de código, y al menos 90% de cobertura de ramas para todo el archivo arimaa.py, no por procedimiento o función**. Esto se verá en la teórica de Testing de caja Blanca y en su clase de laboratorio asociado, utilizando las herramientas `unittest` y `coverage`.

Método de entrega

Se espera que el trabajo grupal lo realicen de forma incremental utilizando un sistema de control de versiones, como Git. Para la entrega, sólo vamos a trabajar con GitLab (pueden ser `git.exactas.uba.ar` si tienen usuario, o deben crearse un usuario en `gitlab.com`). El repositorio debe ser privado, deben estar todos los integrantes del grupo y, además, deben agregar con rol "Developer" un usuario de los docentes:

- Si usan `git.exactas.uba.ar`: Deben agregar al usuario *docentes.ip*
- Si usan `gitlab.com`: Deben agregar al usuario *ip.dc.uba*

La entrega debe ser realizada ***únicamente*** por un integrante del grupo. Debe realizarlo quien haya anotado al grupo en el link para registrar los grupos. La entrega consta de la URL del repositorio y un commit particular. Les docentes descargarán el código para ese commit. En caso que el link o el commit no sean válidos, o no se pueda descargar porque no agregaron el integrante docente con el rol apropiado, el trabajo estará desaprobado, pero tendrán la oportunidad de re-entregar en fecha de recuperatorio.

Antes de enviar el trabajo, se recomienda fuertemente:

- Clonar el repositorio con el link y commit proporcionados. Chequear que ambos son válidos.
- Ejecutar los casos de test de la materia y sus propios casos de test, sobre el repositorio recién clonado.
- Deben incluir todo archivo/carpeta necesario para ejecutar los casos de test.
- Todos los casos de test deben pasar satisfactoriamente, y el archivo con la implementación debe poder cargarse sin generar errores.

Enunciado

Tipos de datos

Asumimos definidas las siguientes constantes:

- `ALTO = 8`, que representa el alto (número de filas) del tablero;
- `ANCHO = 8`, que representa el ancho (número de columnas) del tablero;
- `MAX_PASOS = 4`, que representa el máximo número de pasos por turno.

A lo largo de la especificación, usaremos los siguientes tipos de datos[4]:

```
tipo Figura      = ELEFANTE | CAMELLO | CABALLO | PERRO | GATO | CONEJO
tipo Color       = DORADO | PLATEADO
tipo Casilla     = VACÍA | Pieza
tipo Pieza       = Figura × Color
tipo Tablero     = seq⟨seq⟨Casilla⟩⟩
tipo Acción      = MOVER_A_SECAS | ARRASTRAR | EMPUJAR
tipo Posición    = ℤ × ℤ
tipo Dirección   = NORTE | SUR | ESTE | OESTE
tipo Movimiento  = Acción × Posición × seq⟨ Dirección ⟩
tipo Juego       = Tablero × Color × ℤ
```

Dichos tipos se usan para representar los siguientes elementos del juego:

- El tipo `Figura` se usa para representar cada una de las seis posibles figuras (tipos de piezas) del juego.
- El tipo `Color` se usa para representar los colores de los dos jugadores y sus respectivas piezas.
- El tipo `Casilla` se usa para representar el contenido de una casilla, que puede estar vacía (`VACÍA`) o contener una pieza (que es un valor del tipo `Pieza`).
- El tipo `Pieza` se usa para representar una pieza de una cierta figura y un cierto color, que está dada por una tupla $(figura, color) : \texttt{Figura} \times \texttt{Color}$.
- El tipo `Tablero` se usa para representar el contenido del tablero, que normalmente es una matriz de `ALTO` × `ANCHO` casillas, representada como una secuencia de filas, cada una de las cuales a su vez es una secuencia de casillas.
- El tipo `Acción` se usa para representar la acción de una pieza al moverse, que puede ser un movimiento a secas (`MOVER_A_SECAS`), un movimiento que arrastra una pieza del oponente (`ARRASTRAR`) o un movimiento que empuja una pieza del oponente (`EMPUJAR`).
- El tipo `Posición` se usa para representar una posición, que normalmente es una tupla $(fila, columna)$ donde $0 \leq fila <$ `Alto` y $0 \leq columna <$ `Ancho`.
- El tipo `Dirección` se usa para representar cada una de las cuatro direcciones ortogonales.
- El tipo `Movimiento` se usa para representar un movimiento (de una única pieza, una única casilla en dirección ortogonal). Se representa como una tripla $(acción, origen, pasos) : \texttt{Acción} \times \texttt{Posición} \times seq\langle\texttt{Dirección}\rangle$, donde *pasos* es una lista de direcciones (a las que llamamos "pasos"), con la siguiente interpretación:
  - Si la acción es `MOVER_A_SECAS`, la tripla representa un movimiento a secas. La posición *origen* corresponde a la casilla de origen de la pieza que se mueve y *pasos* es una lista de direcciones, de **exactamente un elemento**, donde dicho paso indica la dirección de movimiento.
  - Si la acción es `ARRASTRAR`, la tripla representa un movimiento que arrastra una pieza del oponente. La posición *origen* corresponde a la casilla de origen de la pieza que se mueve y *pasos* es una lista de direcciones, de **exactamente dos elementos**. El primer paso indica la dirección de movimiento de la pieza que se mueve. El segundo paso indica la dirección de movimiento de la pieza oponente que se arrastra.

[4] Usamos la siguiente sintaxis para describir tipos de datos dados por varias alternativas:
`tipo NOMBRE_DEL_TIPO = Caso_1 | ... | Caso_n`

  - Si la acción es `Empujar`, la tripla representa un movimiento que empuja una pieza del oponente. La posición *origen* corresponde a la casilla de origen de la pieza que se mueve y *pasos* es una lista de direcciones, de **exactamente dos elementos**. El primer paso indica la dirección de movimiento de la pieza que se mueve. El segundo paso indica la dirección de movimiento de la pieza oponente que se empuja.

- El tipo `Juego` se usa para representar el estado actual del juego. Se representa como una tripla de la siguiente forma: $(tablero, turno, pasosConsumidos) : \texttt{Tablero} \times \texttt{Color} \times \mathbb{Z}$, donde *tablero* representa el estado actual del tablero, *turno* es el color del jugador al que le toca jugar actualmente y *pasosConsumidos* es el número de pasos ya efectuados por el jugador en el turno actual.

Tipos de datos y funciones

Normalmente usamos los siguientes nombres de variables:

- i, n para denotar números enteros,
- f para denotar figuras,
- col para denotar colores,
- c para denotar casillas,
- t para denotar tableros,
- pos para denotar posiciones,
- d para denotar direcciones,
- m para denotar movimientos,
- j para denotar juegos.
- Agregamos el sufijo -s para denotar secuencias; por ejemplo, ds puede denotar una lista de direcciones.

Convenciones de notación y funciones primitivas

- Si t es una n-upla, notamos t_i a su i-ésima componente, para cada $1 \leq i \leq n$.
- Si s es una secuencia, notamos $|s|$ a su longitud y $s[i]$ a su i-ésimo elemento para cada $0 \leq i < |s|$.
- Suponemos definida la función `fuerza`(f : `Figura`) : $\mathbb{Z}$ del siguiente modo:

```
fuerza(Conejo)    = 1
fuerza(Gato)      = 2
fuerza(Perro)     = 3
fuerza(Caballo)   = 4
fuerza(Camello)   = 5
fuerza(Elefante)  = 6
```

- Suponemos definida la función `oponente`(col : `Color`) : `Color` que dado un color devuelve el color del oponente, de tal modo que:

```
oponente(Dorado)    = Plateado
oponente(Plateado)  = Dorado
```

- Suponemos definida la función `opuesta`(d : `Dirección`) : `Dirección` que dada una posición devuelve su posición opuesta, de tal modo que:

```
opuesta(Norte)      = Sur
opuesta(Sur)        = Norte
opuesta(Este)       = Oeste
opuesta(Oeste)      = Este
```

- Suponemos definida la constante POSICIONES_TRAMPA : $seq\langle$Posición$\rangle$ del siguiente modo:

  $$\text{POSICIONES_TRAMPA} = [(2, 2), (2, 5), (5, 2), (5, 5)]$$

  cuyos elementos representan las posiciones de las cuatro casillas trampa (C3, F3, C6 y F6).

- Si c es una Casilla no vacía, entonces c es una Pieza. En tal caso, notamos c.figura a su primera componente y c.color a su segunda componente.

- Si p es una Posición, notamos p.fila a su primera componente y p.columna a su segunda componente.

- Si m es un Movimiento, notamos m.acción a su primera componente, m.origen a su segunda componente y m.pasos a su tercera componente.

- Si j es un Juego, notamos j.tablero a su primera componente, j.turno a su segunda componente y j.pasosConsumidos a su tercera componente.

Predicados auxiliares

En esta sección se definen predicados auxiliares para la especificación. Un predicado auxiliar sirve para darle nombre a alguna propiedad que nos interesa en el contexto de un cierto problema, expresada en función de ciertos valores de entrada. Por ejemplo, podemos definir el predicado esPar(n), que es verdadero si y sólo si el entero n es par, del siguiente modo:

```
aux esPar(n : ℤ) := (n mod 2 = 0)
```

Predicados auxiliares de buena formación

```
aux tableroBienFormado(t : Tablero) :=
    |t| = ALTO
  ∧ para todo i:ℤ.(0 ≤ i < |t| → |t[i]| = ANCHO)

aux posiciónEnTablero(pos : Posición) :=
    0 ≤ pos.fila < ALTO
  ∧ 0 ≤ pos.columna < ANCHO

aux movimientoBienFormado(m : Movimiento) :=
    posiciónEnTablero(m.origen)
  ∧ (m.acción = MOVER_A_SECAS → |m.pasos| = 1)
  ∧ (m.acción = ARRASTRAR     → |m.pasos| = 2)
  ∧ (m.acción = EMPUJAR       → |m.pasos| = 2)

aux juegoBienFormado(j : Juego) :=
    tableroBienFormado(j.tablero)
  ∧ 0 ≤ j.pasosConsumidos < MAX_PASOS
```

Predicados auxiliares sobre el tablero y su contenido

```
aux esPosiciónVecinaHacia(pos : Posición, d : Dirección, pos' : Posición) :=
    (d = NORTE → (pos'.fila = pos.fila + 1 ∧ pos'.columna = pos.columna))
  ∧ (d = SUR   → (pos'.fila = pos.fila - 1 ∧ pos'.columna = pos.columna))
  ∧ (d = ESTE  → (pos'.fila = pos.fila     ∧ pos'.columna = pos.columna + 1))
  ∧ (d = OESTE → (pos'.fila = pos.fila     ∧ pos'.columna = pos.columna - 1))

aux hayCasillaVacía(t : Tablero, pos : Posición) :=
    posiciónEnTablero(pos)
  ∧ t[pos.fila][pos.columna] = VACÍA

aux hayFiguraDeColor(t : Tablero, pos : Posición, f : Figura, col : Color) :=
    posiciónEnTablero(pos)
  ∧ ¬hayCasillaVacía(t, pos)
```

```
  ∧ t[pos.fila][pos.columna].figura = f
  ∧ t[pos.fila][pos.columna].color  = col

aux hayCasillaVacíaHacia(t : Tablero, pos : Posición, d : Dirección) :=
  existe pos': Posición . (
      esPosiciónVecinaHacia(pos, d, pos')
    ∧ hayCasillaVacía(t, pos')
  )

aux tablerosCoincidenEnLaPosición(t1, t2 : Tablero, pos : Posición) :=
  t1[pos.fila][pos.columna] = t2[pos.fila][pos.columna]

aux tablerosCoincidenSalvoEnLaPosición(t1, t2 : Tablero, pos : Posición) :=
   para todo pos': Posición .(
    (posiciónEnTablero(pos') ∧ pos' ≠ pos) →
        tablerosCoincidenEnLaPosición(t1, t2, pos')
  )

aux esTableroSacandoPieza(t0, t1 : Tablero, pos : Posición) :=
    tablerosCoincidenSalvoEnLaPosición(t0, t1, pos)
  ∧ hayCasillaVacía(t1, pos)

aux esTableroPoniendoPieza(t0, t1 : Tablero, pos: Posición, f:Figura, col:Color) :=
    tablerosCoincidenSalvoEnLaPosición(t0, t1, pos)
  ∧ hayFiguraDeColor(t1, pos, f, col)

aux esTableroTrasladandoPieza(t0, t1 : Tablero, pos_origen ,pos_destino: Posición) :=
  existe f:Figura .existe col:Color .existe t_intermedio:Tablero . (
      hayFiguraDeColor(t0, pos_origen, f, col)
    ∧ esTableroSacandoPieza(t0, t_intermedio, pos_origen)
    ∧ esTableroPoniendoPieza(t_intermedio, t1, pos_destino, f, col)
  )
```

Predicados auxiliares sobre la lógica del juego

```
aux hayPieza(t : Tablero, pos : Posición) :=
  existe f:Figura. existe col:Color. hayFiguraDeColor(t, pos, f, col)

aux hayPiezaDeColor(t : Tablero, pos : Posición, col : Color) :=
  existe f:Figura. hayFiguraDeColor(t, pos, f, col)

aux hayPiezaAmigaHacia(t : Tablero, pos : Posición, d : Dirección) :=
  existe col:Color. existe pos_amiga: Posición . (
      hayPiezaDeColor(t, pos, col)
    ∧ esPosiciónVecinaHacia(pos, d, pos_amiga)
    ∧ hayPiezaDeColor(t, pos_amiga, col)
  )

aux hayPiezaAmigaVecina(t : Tablero, pos : Posición) :=
  existe d: Dirección . hayPiezaAmigaHacia(t, pos, d)

aux hayPiezaEnemigaMásDébilHacia(t : Tablero, pos : Posición, d : Dirección) :=
  existe f:Figura .existe col:Color. existe pos_enemiga: Posición .existe f_enemiga:Figura. (
      hayFiguraDeColor(t, pos, f, col)
    ∧ esPosiciónVecinaHacia(pos, d, pos_enemiga)
    ∧ hayFiguraDeColor(t, pos_enemiga, f_enemiga, oponente(col))
    ∧ fuerza(f) > fuerza(f_enemiga)
```

```
  )

aux hayPiezaEnemigaMásFuerteHacia(t : Tablero, pos : Posición, d : Dirección) :=
  existe f: Figura. existe col: Color. existe pos_enemiga: Posición. existe f_enemiga: Figura. (
      hayFiguraDeColor(t, pos, f, col)
    ∧ esPosiciónVecinaHacia(pos, d, pos_enemiga)
    ∧ hayFiguraDeColor(t, pos_enemiga, f_enemiga, oponente(col))
    ∧ fuerza(f) < fuerza(f_enemiga)
  )

aux hayPiezaEnemigaMásFuerteVecina(t : Tablero, pos : Posición) :=
  existe d: Dirección. hayPiezaEnemigaMásFuerteHacia(t, pos, d)

aux hayPiezaCongelada(t : Tablero, pos : Posición) :=
    hayPieza(t, pos)
  ∧ hayPiezaEnemigaMásFuerteVecina(t, pos)
  ∧ ¬hayPiezaAmigaVecina(t, pos)
```

Predicados auxiliares para determinar si es posible realizar un movimiento

```
aux direcciónDeMovimientoHabilitada(t : Tablero, pos : Posición, d : Dirección) :=
  existe f: Figura. existe col: Color. (
      hayFiguraDeColor(t, pos, f, col)
    ∧ ((f = Conejo ∧ col = Dorado)   → d ≠ Sur)
    ∧ ((f = Conejo ∧ col = Plateado) → d ≠ Norte)
  )

aux puedeMoverASecas(t : Tablero, pos : Posición, d : Dirección) :=
    hayCasillaVacíaHacia(t, pos, d)

aux puedeArrastrar(t : Tablero, pos : Posición, d, d_arrastre : Dirección) :=
    hayCasillaVacíaHacia(t, pos, d)
  ∧ hayPiezaEnemigaMásDébilHacia(t, pos, opuesta(d_arrastre))

aux puedeEmpujar(t : Tablero, pos : Posición, d, d_empuje : Dirección) :=
    hayPiezaEnemigaMásDébilHacia(t, pos, d)
  ∧ existe pos_enemiga: Posición. (
        esPosiciónVecinaHacia(pos, d, pos_enemiga)
      ∧ hayCasillaVacíaHacia(t, pos_enemiga, d_empuje)
    )

aux puedeRealizarAcción(t : Tablero, m : Movimiento) :=
    (m.acción = Mover_a_secas →
       puedeMoverASecas(t, m.origen, m.pasos[0]))
  ∧ (m.acción = Arrastrar →
       puedeArrastrar(t, m.origen, m.pasos[0], m.pasos[1]))
  ∧ (m.acción = Empujar →
       puedeEmpujar(t, m.origen, m.pasos[0], m.pasos[1])

aux puedeRealizarMovimiento(j : Juego, m : Movimiento) :=
    hayPiezaDeColor(j.tablero, m.origen, j.turno)
  ∧ ¬hayPiezaCongelada(j.tablero, m.origen)
  ∧ direcciónDeMovimientoHabilitada(j.tablero, m.origen, m.pasos[0])
  ∧ j.pasosConsumidos + |m.pasos| ≤ Max_pasos
  ∧ puedeRealizarAcción(j.tablero, m)
```

Predicados auxiliares para "activar" las casillas trampa

```
aux esTableroDespuésDeActivarTrampa(t0, t1 : Tablero, pos_trampa : Posición) :=
    ( hayPiezaAmigaVecina(t0, pos_trampa) → t1 = t0)
  ∧ (¬hayPiezaAmigaVecina(t0, pos_trampa) → esTableroSacandoPieza(t0, t1, pos_trampa))

aux esTableroDespuésDeActivarTodasLasTrampas(t0, t1 : Tablero) :=
  existe t_intermedio1, t_intermedio2, t_intermedio3 : Tablero. (
      esTableroDespuésDeActivarTrampa(t0,           t_intermedio1, POSICIONES_TRAMPA[0])
    ∧ esTableroDespuésDeActivarTrampa(t_intermedio1, t_intermedio2, POSICIONES_TRAMPA[1])
    ∧ esTableroDespuésDeActivarTrampa(t_intermedio2, t_intermedio3, POSICIONES_TRAMPA[2])
    ∧ esTableroDespuésDeActivarTrampa(t_intermedio3, t1,            POSICIONES_TRAMPA[3])
  )
```

Predicados auxiliares para determinar el estado del juego después de un movimiento

```
aux tableroPostMoverASecas(t0, t1 : Tablero, pos : Posición, d : Dirección) :=
  existe pos_destino : Posición . existe t_intermedio : Tablero. (
      esPosiciónVecinaHacia(pos, d, pos_destino)
    ∧ esTableroTrasladandoPieza(t0, t_intermedio, pos, pos_destino)
    ∧ esTableroDespuésDeActivarTodasLasTrampas(t_intermedio, t1)
  )

aux tableroPostArrastrar(t0, t1 : Tablero, pos : Posición, d, d_arrastre : Dirección) :=
  existe pos_hueco, pos_enemiga : Posición . existe t_intermedio1, t_intermedio2 : Tablero. (
      esPosiciónVecinaHacia(pos, d, pos_hueco)
    ∧ esPosiciónVecinaHacia(pos, opuesta(d_arrastre), pos_enemiga)
    ∧ esTableroTrasladandoPieza(t0, t_intermedio1, pos, pos_hueco)
    ∧ esTableroTrasladandoPieza(t_intermedio1, t_intermedio2, pos_enemiga, pos)
    ∧ esTableroDespuésDeActivarTodasLasTrampas(t_intermedio2, t1)
  )

aux tableroPostEmpujar(t0, t1 : Tablero, pos : Posición, d, d_empuje : Dirección) :=
  existe pos_enemiga, pos_hueco : Posición . existe t_intermedio1, t_intermedio2 : Tablero. (
      esPosiciónVecinaHacia(pos, d, pos_enemiga)
    ∧ esPosiciónVecinaHacia(pos_enemiga, d_empuje, pos_hueco)
    ∧ esTableroTrasladandoPieza(t0, t_intermedio1, pos_enemiga, pos_hueco)
    ∧ esTableroTrasladandoPieza(t_intermedio1, t_intermedio2, pos, pos_enemiga)
    ∧ esTableroDespuésDeActivarTodasLasTrampas(t_intermedio2, t1)
  )

aux tableroPostMovimiento(t0, t1 : Tablero, m : Movimiento) :=
    (m.acción = MOVER_A_SECAS →
        tableroPostMoverASecas(t0, t1, m.origen, m.pasos[0]))
  ∧ (m.acción = ARRASTRAR →
        tableroPostArrastrar(t0, t1, m.origen, m.pasos[0], m.pasos[1]))
  ∧ (m.acción = EMPUJAR →
        tableroPostEmpujar(t0, t1, m.origen, m.pasos[0], m.pasos[1]))

aux hayQueCambiarDeTurno(j : Juego, m : Movimiento) :=
  j.pasosConsumidos + |m.pasos| = MAX_PASOS

aux juegoConTurnoPosiblementeAvanzado(j0, j1 : Juego, m : Movimiento) :=
    (hayQueCambiarDeTurno(j0, m) →
        (j1.turno = oponente(j0.turno) ∧ j1.pasosConsumidos = 0))
  ∧ (¬hayQueCambiarDeTurno(j0, m) →
        (j1.turno = j0.turno ∧ j1.pasosConsumidos = j0.pasosConsumidos + |m.pasos|))

aux juegoPostMovimiento(j0, j1 : Juego, m : Movimiento) :=
    tableroPostMovimiento(j0.tablero, j1.tablero, m)
```

```
  ∧ juegoConTurnoPosiblementeAvanzado(j0, j1, m)
```

Predicados auxiliares para determinar el ganador del juego

```
aux esInicioDeTurno(j : Juego, col : Color) :=
    j.turno = col
  ∧ j.pasosConsumidos = 0

aux hayConejoEnTablero(j : Juego, col : Color) :=
  existe pos: Posición . hayFiguraDeColor(j.tablero, pos, Conejo, col)

aux posiciónEnFilaInicial(pos : Posición, col : Color) :=
    (col = Dorado   → pos.fila = 0)
  ∧ (col = Plateado → pos.fila = 7)

aux hayConejoEnFilaInicialOponente(j : Juego, col : Color) :=
  existe pos: Posición .(
    posiciónEnFilaInicial(pos, oponente(col))
  ∧ hayFiguraDeColor(j.tablero, pos, Conejo, col)
  )

aux reúneCondicionesDeGanar(j : Juego, col : Color) :=
    hayConejoEnFilaInicialOponente(j, col)
  ∨ ¬hayConejoEnTablero(j, oponente(col))

aux reúneCondicionesDeGanarExclusivas(j : Juego, col : Color) :=
    reúneCondicionesDeGanar(col)
  ∧ ¬reúneCondicionesDeGanar(j, oponente(col))

aux esJugadorGanador(j : Juego, col : Color) :=
    (esInicioDeTurno(j, oponente(col)) ∧ reúneCondicionesDeGanar(j, col))
  ∨ (esInicioDeTurno(j, col)           ∧ reúneCondicionesDeGanarExclusivas(j, col))

aux hayGanadorDeterminado(j : Juego) :=
  existe col:Color. esJugadorGanador(j, col)
```

Ejercicios

Se pide implementar los siguientes problemas.

Ejercicio 1

```
problema tablero_bien_formado(in t : Tablero) : Bool {
  requiere { True }
  asegura  { res = TRUE ⟷ tableroBienFormado(t) }
}

problema posición_en_tablero(in pos : Posición) : Bool {
  requiere { True }
  asegura  { res = TRUE ⟷ posiciónEnTablero(pos) }
}

problema movimiento_bien_formado(in m : Movimiento) : Bool {
  requiere { True }
  asegura  { res = TRUE ⟷ movimientoBienFormado(m) }
}

problema juego_bien_formado(in j : Juego) : Bool {
  requiere { True }
  asegura  { res = TRUE ⟷ juegoBienFormado(j) }
}
```

Ejercicio 2

```
problema posición_vecina_hacia(in pos : Posición , in d : Dirección ) : Posición {
  requiere { True }
  asegura  { esPosiciónVecinaHacia(pos, d, res) }
}

problema casilla_en_posición(in t : Tablero , in pos : Posición ) : Casilla {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  asegura  { res = t[pos.fila][pos.columna] }
}

problema hay_casilla_vacía(in t : Tablero , in pos : Posición ) : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  asegura  { res = TRUE ⟷ hayCasillaVacía(t, pos) }
}

problema hay_pieza_de_color(in t : Tablero , in pos : Posición , in col : Color) : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  asegura  { res = TRUE ⟷ hayPiezaDeColor(t, pos, col) }
}
```

Ejercicio 3

```
problema hay_casilla_vacía_hacia(in t : Tablero , in pos : Posición , in d : Dirección )
                                                                          : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  asegura  { res = TRUE ⟷ hayCasillaVacíaHacia(t, pos, d) }
}

problema hay_pieza_amiga_vecina(in t : Tablero , in pos : Posición ) : Bool {
```

```
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  requiere { hayPieza(t, pos) }
  asegura  { res = TRUE ⟷ hayPiezaAmigaVecina(t, pos) }
}

problema hay_pieza_enemiga_más_débil_hacia(in t : Tablero, in pos : Posición ,
                                           in d : Dirección ) : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  requiere { hayPieza(t, pos) }
  asegura  { res = TRUE ⟷ hayPiezaEnemigaMásDébilHacia(t, pos, d) }
}

problema hay_pieza_enemiga_más_fuerte_vecina(in t : Tablero, in pos : Posición ) : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  requiere { hayPieza(t, pos) }
  asegura  { res = TRUE ⟷ hayPiezaEnemigaMásFuerteVecina(t, pos) }
}

problema hay_pieza_congelada(in t : Tablero, in pos : Posición ) : Bool {
  requiere { tableroBienFormado(t) ∧ posiciónEnTablero(pos) }
  asegura  { res = TRUE ⟷ hayPiezaCongelada(t, pos) }
}
```

Ejercicio 4

```
problema puede_realizar_movimiento(in j : Juego, in m : Movimiento) : Bool {
  requiere { juegoBienFormado(j) ∧ movimientoBienFormado(m) }
  asegura  { res = TRUE ⟷ puedeRealizarMovimiento(j, m) }
}
```

Ejercicio 5

```
problema activar_trampas(in t : Tablero) : Tablero {
  requiere { tableroBienFormado(t) }
  asegura  { tableroBienFormado(res) }
  asegura  { esTableroDespuésDeActivarTodasLasTrampas(t, res) }
}
```

Ejercicio 6

```
problema realizar_movimiento(in j : Juego, in m : Movimiento) : Juego {
  requiere { juegoBienFormado(j) ∧ movimientoBienFormado(m) }
  requiere { puedeRealizarMovimiento(j, m) }
  asegura  { juegoBienFormado(res) }
  asegura  { juegoPostMovimiento(j, res, m) }
}
```

Ejercicio 7

```
problema hay_ganador_determinado(in j : Juego) : Bool {
  requiere { juegoBienFormado(j) }
  asegura  { res = TRUE ⟷ hayGanadorDeterminado(j) }
}

problema ganador_del_juego(in j : Juego) : Color {
  requiere { juegoBienFormado(j) }
```

```
  requiere { hayGanadorDeterminado(j) }
  asegura  { esJugadorGanador(j, res) }
}
```

Tipos de datos disponibles en Python

Se proveen ya tipos de datos definidos en Python. Se ilustran con ejemplos.

Figuras

```
>>> [ELEFANTE, CAMELLO, CABALLO, PERRO, GATO, CONEJO]
[ELEFANTE, CAMELLO, CABALLO, PERRO, GATO, CONEJO]
```

Colores

```
>>> [DORADO, PLATEADO]
[DORADO, PLATEADO]

>>> oponente(DORADO)
PLATEADO
```

Piezas y casillas

```
>>> lista_de_casillas = [VACIA, Pieza(ELEFANTE, DORADO)]

>>> lista_de_casillas[0]
VACIA

>>> lista_de_casillas[1]
Pieza(figura=ELEFANTE, color=DORADO)

>>> lista_de_casillas[1].figura
ELEFANTE

>>> lista_de_casillas[1].color
DORADO
```

Acciones

```
>>> lista_de_acciones = [MOVER_A_SECAS, ARRASTRAR, EMPUJAR]
[MOVER_A_SECAS, ARRASTRAR, EMPUJAR]
```

Posiciones

```
>>> pos = Posicion(1, 5)

>>> pos.fila
1

>>> pos.columna
5
```

Direcciones

```
>>> lista_de_direcciones = [NORTE, SUR, ESTE, OESTE]

>>> opuesta(OESTE)
ESTE
```

Movimientos

```
>>> m = Movimiento(ARRASTRAR, Posicion(1, 5), [ESTE, SUR])

>>> m.accion
ARRASTRAR
```

```
>>> m.origen
Posicion(fila=1, columna=5)

>>> m.pasos
[ESTE, SUR]

>>> m.pasos[1]
SUR
```

Tablero

Se provee una función `leer_tablero` que recibe una lista de *strings* y devuelve un tablero. Cada *string* representa una de las filas del tablero. Cada caracter de la fila representa una de las celdas de dicha fila. Cada celda puede estar vacía, lo que se representa con un *guión* (`-`), o contener una pieza. Las piezas se representan con letras del alfabeto. Las letras mayúsculas representan piezas doradas y las minúsculas piezas plateadas, y se usan las siguientes asociaciones:

- `E` / `e`: elefante dorado / plateado
- `M` / `m`: camello dorado / plateado
- `H` / `h`: caballo dorado / plateado
- `D` / `d`: perro dorado / plateado
- `C` / `c`: gato dorado / plateado
- `R` / `r`: conejo dorado / plateado

Por ejemplo:

```
>>> tablero = leer_tablero([
      'emhhddcc', #7
      'rrrrrrrr', #6
      '--------', #5
      '--------', #4
      '--------', #3
      '--------', #2
      'RRRRRRRR', #1
      'HCDMEDCH', #0
      #01234567
    ])

>>> tablero[7][0]
Pieza(figura=ELEFANTE, color=PLATEADO)

>>> tablero[4][4]
VACIA
```

Juegos

```
>>> juego = Juego(tablero, DORADO, 1)

>>> juego.tablero[7][0]
Pieza(figura=ELEFANTE, color=PLATEADO)

>>> juego.turno
DORADO

>>> juego.pasos_consumidos
1
```